

Tema 7. Lectura e escritura de información.

Parte 2

Uso de ficheiros

1.	<u>Ficheiros</u>	3
1.1	Ficheiros de datos	3
1.2	Tipos de acceso a ficheiros	4
1.3	Tipos de ficheiros	5
1.4	Operacións sobre ficheiros	5
2.	<u>Formas de traballar con ficheiros en Java</u>	6
3.	<u>A clase File de Java</u>	7
3.1	Erros ao traballar con ficheiros	9
3.2	Xestión de ficheiros e directorios	10
3.3	Interface FilenameFilter	14
4.	<u>Manipulación de ficheiros de texto</u>	16
4.1	Apertura de ficheiro de texto	16
4.2	Peche de ficheiro de texto	17
4.3	Lectura de ficheiros de texto	18
4.4	Escritura en ficheiros de texto	21
5.	<u>Manipulación de ficheiros binarios</u>	24
5.1	Apertura de ficheiros binarios	24
5.2	Peche de ficheiros binarios	25
5.3	Escritura en ficheiros binarios	26
5.4	Lectura de ficheiros binarios	28
6.	<u>Ficheiros de acceso aleatorio</u>	30
7.	<u>Obxectos en ficheiros binarios. Serialización</u>	37
7.1	Problema cos ficheiros de obxectos	40

1. Ficheiros

Ata este momento, as estruturas de datos utilizadas para o almacenamento de información residían en memoria. Isto ten dúas limitacións importantes:

- Os datos non se almacenan de forma permanente. Unha vez que se sae do programa e volvemos a executalo non podemos acceder á información xerada na execución anterior do programa.
- A cantidade dos datos non pode ser moi grande debido á limitación do tamaño da memoria.

Para evitar estes inconvenientes existen unhas estruturas que utilizan a memoria secundaria para o almacenamento da información as cales se identifican baixo un nome e unha extensión. Estas estruturas denomínanse **ficheiros**.

Os ficheiros teñen un nome e están localizados en directorios ou cartafoles, o nome debe ser único nese directorio; non pode haber dous ficheiros co mesmo nome nese directorio. Por convención teñen extensións diferentes que adoitan ser 3 caracteres (PDF, DOC, GIF, ...) e permítenos coñecer o tipo de arquivo.

1.1 Ficheiros de datos

Existe o concepto de **ficheiros de datos** podemos definilos coma un conxunto de datos, sobre un mesmo tema, tratado coma unha unidade de almacenamento e organizado de forma estruturada para a procura dun dato individual.

Podemos considerar o ficheiro como unha estrutura dinámica no senso de que o seu tamaño pode variar durante a execución do programa dependendo da cantidade de datos que teña.

A información, ou datos do ficheiro, atópase almacenada seguindo unha determinada estrutura. A esta estrutura denomínaselle rexistro.

Podemos definir un **rexistro** coma un conxunto de datos que forman unha unidade de información e que se manipulan coma un bloque. Cada rexistro está composto de unidades elementais de información máis pequenas que se denominan **campos**.

Na seguinte imaxe vemos un exemplo dun ficheiro que almacena datos de empregados. Cada unha das liñas do ficheiro correspóndese cun rexistro, que á súa vez componse de varios campos, onde cada un fai referencia á seguinte información: nome, dni, idade, departamento.

En Java, non existen **registros** como tal en ficheiros ao estilo dunha estrutura de datos específica (como en outras linguaxes que teñen registros ou structs). O que hai que facer é simulalos mediante a escrita e lectura de datos en formato texto ou binario, estruturándoos de xeito que poidan ser interpretados posteriormente. Por exemplo, gardar atributos en liñas distintas ou en unha única liña con delimitadores, e logo analizando esa información ao lela.

Nome	DNI	Idade	Departamento
------	-----	-------	--------------



Ficheiro: empregados.dat

Pedro	11111111-A	25	RR.HH.
Ana	22222222-B	32	Programación
Xan	33333333-C	50	Análise&Deseño
Eduardo	44444444-D	29	Administración
Francisco	55555555-E	33	Recepción
Marta	66666666-F	35	Análise&Deseño
Sofía	77777777-G	47	RR.HH

1.2 Tipos de acceso a ficheiros

Existen dúas formas de acceder á información almacenada nun ficheiro: acceso secuencial e acceso directo ou aleatorio:

- **Acceso secuencial:** os datos ou os rexistros son lidos e escritos en orde, do mesmo xeito que se fai nunha vella cinta de vídeo. Se se desexa acceder a un dato ou un rexistro que está cara ao medio do ficheiro, é necesario ler todos os anteriores antes. A escritura de datos farase a partir dos últimos datos escritos, non se poden facer insercións entre os datos xa escritos.

Os ficheiros secuenciais úsanse normalmente en aplicacións por lotes, por exemplo, na copia de seguridade dos datos ou copias de seguridade, e son óptimos nestas aplicacións se procesa todos os rexistros. A **vantaxe** destes ficheiros é a rápida accesibilidade ao seguinte rexistro (son rápidos cando os rexistros se acceden de forma secuencial) e iso fan un mellor uso do espazo. Tamén son fáciles de usar e aplicar.

A **desvantaxe** é que non podes acceder directamente a un rexistro en particular, debes ler antes todos os anteriores; é dicir, non soporta o acceso aleatorio. Outra **desvantaxe** é o proceso de actualización, a maioría dos ficheiros secuenciais non se poden actualizar, será necesario reescribilos por completo. Para aplicacións interactivas que inclúen peticións ou actualizacións individuais de rexistros, os ficheiros secuenciais ofrecen un rendemento deficiente.

- **Acceso directo ou aleatorio:** permite o acceso directo a un rexistro de datos sen necesidade de ler os anteriores e pódese acceder á información en calquera orde. Os datos almacénanse en rexistros de tamaño coñecido, podemos movernos dun rexistro a outro aleatoriamente para ler ou modificalos.

Unha das principais **vantaxes** dos ficheiros aleatorios é o acceso rápido a unha posición determinada para ler ou escribir un rexistro. A gran **desvantaxe** é establecer a relación

entre a posición ocupada polo rexistro e o seu contido; xa que ás veces ao aplicar a función de conversión para obter a posición obtéñense posicións ocupadas e tense que percorrer á área de excedentes. Outro **inconveniente** é que parte do espazo asignado ao ficheiro pódese desperdiciar, xa que poden producirse ocos (posicións desocupadas) entre un rexistro e outro.

En Java, o acceso secuencial máis común en ficheiros pode ser binario ou con caracteres. Para o acceso binario: úsanse as clases **FileInputStream** e **FileOutputStream**; para acceder a caracteres (texto) úsanse as clases **FileReader** e **FileWriter**. No acceso aleatorio, úsase a clase **RandomAccessFile**.

1.3 Tipos de ficheiros

A clasificación por tipo de ficheiro baséase na forma na que se garda a información dentro do ficheiro. Existen dous tipos principais de ficheiros:

- **Ficheiros de texto:** a información almacénase en formato de caracteres, tal e como se vería por pantalla. Por exemplo, o valor enteiro 25 transformarase nos caracteres ‘2’ e ‘5’ que serán almacenados no ficheiro.
- **Ficheiros binarios:** as secuencias de elementos almacenadas no ficheiro son dun determinado tipo de datos. É dicir, a información gárdase no ficheiro tal e como está en memoria principal, polo tanto, estará composta por uns e ceros. Por exemplo, o valor enteiro 25 gardarase como a secuencia binaria 00011001.

Outra clasificación que se pode facer sobre os ficheiros é a que distingue entre:

- **Ficheiros físicos:** é o ficheiro que reconece o sistema operativo e que está almacenado en memoria secundaria baixo un nome e unha extensión.
- **Ficheiros lóxicos:** o programa debe poder facer referencia ao ficheiro físico para traballar con el. Desta maneira as linguaxes de programación proporcionan mecanismos para poder crear un tipo de dato de **tipo ficheiro**. En Java, se queremos crear unha variable que apunta a un ficheiro físico debemos utilizar a **clase File**. Desta forma o ficheiro lóxico será un **obxecto** da clase **File**.

1.4 Operacións sobre ficheiros

As operacións básicas que se realizan en calquera ficheiro independentemente da forma de acceso a el son os seguintes:

- **Creación do ficheiro.** O ficheiro créase no disco cun nome que é usado para acceder a el. A creación é un proceso que se realiza unha vez.
- **Apertura do ficheiro.** Para que un programa poida operar cun ficheiro, o primeiro que ten que facer é abrílo. O programa usará algún método para identificar o ficheiro co que desexa traballar, por exemplo, asignar a unha variable o descritor do ficheiro.
- **Pechado do ficheiro.** O ficheiro debe estar pechado cando o programa non o vai usar. Normalmente é a última instrución do programa.
- **Lectura dos datos do ficheiro.** Este proceso implica transferir información do ficheiro para a memoria principal, xeralmente a través dalgunha variable ou variables do noso programa no que se depositarán os datos extraídos do ficheiro.

- **Escritura de datos no ficheiro.** Neste caso, o proceso implica a transferencia de información da memoria (mediante as variables do programa) ao ficheiro.

Normalmente, as operacións típicas que se realizan nun ficheiro de datos cando se abren son as seguintes:

- **Altas:** consiste en engadir un novo rexistro ao ficheiro.
- **Baixas:** consiste en eliminar un rexistro existente do ficheiro. A eliminación pode ser lóxica, cambiando o valor dalgún campo do rexistro que usamos para controlar a situación; ou física, eliminando fisicamente o rexistro do ficheiro. O borrado consiste moitas veces en volver escribir o ficheiro noutro ficheiro sen os datos que se que- ren eliminar e, a continuación, cambiarlle o nome ao ficheiro orixinal.
- **Modificacións:** consiste en cambiar o contido dun rexistro. Antes de facer a modifi- cación será necesario localizar o rexistro que se modifica dentro do ficheiro; e unha vez localizados, realízanse os cambios e reescíbese o rexistro.
- **Consultas:** consiste en buscar no ficheiro un rexistro específico.

2. Formas de traballar con ficheiros en Java

As formas xerais de traballar con ficheiros en Java inclúen:

- Paquete **java.io**: É a maneira clásica de traballo con ficheiros en Java.
 - Clase **File**: para crear, borrar, renombrar ficheiros e directorios, obter propiedades.
 - **Streams** de entrada/salida (InputStream, OutputStream, Reader, Writer): para ler e escribir datos en ficheiros, xa sexa en bytes ou caracteres.
 - Uso común: FileReader/BufferedReader para ler texto, FileWriter/BufferedWriter para escritura, PrintWriter para impresión formateada.
- Paquete **java.nio** (desde Java 7)
 - Clases **Path** e **Paths** para representar rutas.
 - Clase **Files** con métodos estáticos para crear, ler, escribir, copiar, borrar ficheiros de forma sinxela e eficiente.
 - Mellora o rendemento e facilita o traballo con ficheiros e directorios.
- Lambdas e Streams (desde Java 8)
 - **Files.lines(Path)** devolve un **Stream<String>** que permite procesar liñas de texto cunha expresión lambda ou outros métodos funcionais.
 - Facilita o tratamento funcional das liñas do ficheiro, como filtros, transformacións, impresión.

Característica	java.io	java.nio	Lambdas / Streams
Enfoque	Imperativo	API moderna, máis eficiente	Declarativo / funcional
Representación de ficheiros	File	Path	Path + Stream<String>
Rendemento	Medio	Alto	Alto (dependendo do uso)
Verbosidade	Alta	Media	Baixa

Compatibilidade	Todas as versións	Java 7+	Java 8+
Ideal para	Ler/escribir básico	Xestión avanzada de ficheiros	Procesar texto ou datos

Polo momento imos a estudar a forma clásica de traballo con ficheiros de Java, o paquete **java.io**.

Existe un enorme número de aplicacións que usan a antiga API (o que se chama legacy). O termo legacy refírese a un sistema, código ou tecnoloxía antiga que aínda se utiliza, normalmente porque funciona e resulta custoso ou arriscado substituíla. Oracle non ten plan de eliminar ou desaconsellar o seu uso nas vindeiras versións de Java xa que:

- Hai millóns de programas que a usan.
- O custo de obrigar a migrar todo ese código sería enorme.
- Moitas librarías e frameworks aínda dependen dela.

É por iso, porque podemos atopar infinidade de código en funcionamento ao que lle temos que dar mantemento (ou facer migración), e que non vai desaparecer de xeito inminente, polo que temos que coñecer esta API. Ademais esta API é a única compatible con todas as versións de Java.

3. A clase File de Java

A clase **File** atópase no paquete **java.io**. Esta clase pode representar tanto a un ficheiro como a un directorio e permite realizar operacións sobre o ficheiro a nivel do sistema de arquivos: tal como listar os arquivos, crear directorios, borrar ficheiros, modificar o seu nome, etc.

Os sistemas operativos utilizan un nome de ruta para facer referencia aos arquivos e directorios que dependen do propio sistema operativo. Esta clase permite obter unha representación abstracta da ruta que é independente do sistema.

Este nome de ruta pode ser de dous tipos:

- **Ruta absoluta:** neste caso a ruta contén a información completa para poder acceder ao arquivo. Polo tanto, nesta ruta debe indicarse antes do ficheiro o directorio raíz. Podemos dicir que a ruta absoluta está formada por un prefixo, un listado de directorios e o nome do directorio ou arquivo ao que queremos acceder.
 - No caso do sistema operativo **Windows**, o prefixo está composto polo nome da unidade, seguido do carácter “.” e a continuación ou ben “/” ou “\”. O listado de directorios para chegar ao destino sepáranse ben con “/” ou “\”. A continuación móstrase a ruta absoluta para chegar ao arquivo `arquivo.txt`.

```
C:\\directorio1\\directorio11\\arquivo.txt
```

- No caso do sistema operativo **Unix**, o prefixo está composto sempre por “/” que representa a raíz do sistema de directorios. O listado de directorios sepáranse polo carácter “/”: A continuación móstrase a ruta absoluta para chegar ao arquivo `arquivo.txt`.

```
/home/user1/directorio1/directorio11/arquivo.txt
```

- **Ruta relativa:** indica a localización dun ficheiro ou directorio en relación co directorio actual do programa, en lugar de indicar a ruta completa desde a raíz do sistema: Se o ficheiro está non mesmo directorio que o programa, neste caso só sería necesario indicar o nome do directorio ou arquivo ao cal queremos acceder. Usando a ruta relativa, o arquivo indicado procúrase a partir do directorio actual, que é o directorio do arquivo Java no que se está traballando. Se desde o directorio do arquivo Java, queremos acceder ao ficheiro exemplo.txt, que está na subcarpeta datos:

```
datos/exemplo.txt
```

Para crear un obxecto de tipo arquivo a partir do sistema de ficheiros do sistema operativo, a clase **File** proporciona 3 construtores:

- **File(String directorioeficheiro):** Un construtor que ten **un só argumento** de tipo **cadea** que representa a ruta ao sistema de arquivos.

```
//Ruta absoluta Windows
File arquivoW = new File("C:\\\\info\\archivos\\archivo.txt");
//Ruta absoluta Unix
File arquivoU = new File("/home/info/archivos/archivo.txt");
//Ruta relativa
File arquivo = new File("archivo.txt");
```

- **File(String directorio, String ficheiro) e File(File directorio, String ficheiro):** Dous construtores nos que se deben especificar **dous argumentos**:
 - O primeiro argumento pode ser de tipo **File** ou unha **cadea** de caracteres. En ambos casos representa á ruta pai.
 - O segundo argumento é de tipo **cadea** e representa á ruta filla. Esta ruta filla é unha segunda ruta que se define a partir da posición da primeira ruta, da ruta pai.

```
String directorio = "C:/EXERCICIOS/UNIDADE1":
File ficheiro2 = new File(directorio, "ejemplo2.txt");
File direc = new File (directorio);
File ficheiro3 = new File(direc, "ejemplo3.txt");
```

A Máquina Virtual de Java permítenos executar código escrito en Java independentemente da plataforma desde o que se realiza a execución. Desta forma, en Java podemos escribir programas que se poden executar en calquera plataforma. Esta grande versatilidade de Java a debemos ter en conta cando especificamos a ruta do arquivo.

Se queremos que a execución do noso programa non dependa tan directamente do sistema operativo a utilizar podemos usar o separador “/” que é válido tanto para Windows coma para Linux ou tamén podemos facer uso das variables estáticas que posúe File.

Variable estática	Descrición
<code>public static final char separatorChar</code>	Representa ao carácter separador de nomes de arquivos e carpetas.
<code>public static final String separator</code>	Como o anterior pero en forma de String.

```
String ruta="UsersPublic/Documents/datos.txt";
ruta=ruta.replace("/", File.separator);
```


Podemos clasificar toda a lista de métodos da clase **File** do seguinte xeito:

- Métodos xerais: que proporcionan información xeral sobre o obxecto. Permítennos saber entre outras cousas se o ficheiro existe, se se pode ler, se está oculto ou se dous arquivos son iguais.
- Métodos de directorio: permiten facer operacións sobre un directorio. Por exemplo, podemos crear un directorio dentro del ou obter a lista dos arquivos que contén.
- Métodos de arquivo: permiten facer operacións sobre un arquivo. Entre outros métodos, hai métodos para renomear un arquivo, para borrarlo ou para saber a súa lonxitude.

Os principais métodos da clase File son:

Método	Función
String[] list()	Devolve un array de String cos nomes de ficheiros e directorios asociados ao obxecto File
File [] listFiles()	Devolve un array de obxectos File contendo os ficheiros que estean dentro do directorio representado polo obxecto File
String getName()	Devolve o nome do ficheiro ou directorio
String getPath()	Devolve o camiño relativo
String getAbsolutePath()	Devolve o camiño absoluto do ficheiro/ directorio
boolean exists()	Devolve true se o ficheiro/directorio existe
boolean canWrite()	Devolve true se o ficheiro se pode escribir
boolean canRead()	Devolve true se o ficheiro se pode ler
boolean isFile()	Devolve true se o obxecto File corresponde a un ficheiro normal
boolean isDirectory()	Devolve true se o obxecto File corresponde a un directorio
long length ()	Devolve o tamaño do ficheiro en bytes
boolean mkdir()	Crea un directorio co nome indicado na creación do obxecto File. Só se creará se non existe
boolean mkdirs()	Crea o directorio especificado e todos os directorios pais necesarios. É útil cando queremos asegurarnos de que toda a ruta existe
boolean renameTo(File novonome)	Renomea o ficheiro representado polo obxecto File asignándolle novonome
boolean delete()	Borra o ficheiro ou directorio asociado ao obxecto File
boolean createNewFile()	Crea un novo ficheiro, vacío, asociado a File se y só se non existe un ficheiro con dito nome
String getParent()	Devolve o nome do directorio pai, ou null se non existe

3.1 Erros ao traballar con ficheiros

Traballar con ficheiros en Java implica manexar excepcións con frecuencia, xa que hai moitos factores externos (permisos, disco, rutas, etc.) que poden fallar.

As excepcións máis comúns que poden ocorrer ao crear e traballar con ficheiros son:

- **IOException:** Do paquete *java.io*. É a máis xeral das excepcións de entrada/saída. Ocorre cando hai un erro de E/S (lectura, escritura, creación, borrado, etc.). Pode deberse a:
 - O ficheiro non pode ser accedido.
 - Problemas de permisos.

- O dispositivo está cheo ou non dispoñible.

```
try {
    ficheiro.createNewFile();
} catch (IOException e) {
    System.out.println("Erro de entrada/saída: " + e.getMessage());
}
```

- **FileNotFoundException:** Subclase de **IOException**. Prodúcese cando se intenta abrir un ficheiro que non existe ou non se pode acceder por permisos. É moi habitual ao usar **FileReader**, **FileInputStream**, etc.

```
try {
    FileReader fr = new FileReader("datos.txt");
} catch (FileNotFoundException e) {
    System.out.println("O ficheiro non foi atopado.");
}
```

- **SecurityException:** Lánzase cando o sistema de seguridade (SecurityManager) denega acceso ao ficheiro ou directorio. Pode ocorrer en aplicacións con restricións.
- **EOFException:** Subclase de **IOException**. Significa “End Of File”, fin de ficheiro inesperada ao ler datos dun fluxo como **InputStream** ou **DataInputStream**.

3.2 Xestión de ficheiros e directorios

Podemos crear un ficheiro do seguinte modo:

- Creamos o obxecto que encapsula o ficheiro, por exemplo, supoñendo que imos crear un ficheiro chamado **miFichero.txt**, no cartafol **C:\\proba**, faríamos:

```
File ficheiro = new File("c:\\proba\\miFichero.txt");
```

- A partir do obxecto **File** creamos o ficheiro fisicamente, coa seguinte instrución, que devolve un boolean con valor true se se creou correctamente, ou false se non se puido crear:

```
ficheiro.createNewFile()
```

Para borrar un ficheiro, podemos usar a clase **File**, comprobando previamente se existe, do seguinte modo:

- Fixamos o nome do cartafol e do ficheiro con:

```
File ficheiro = new File("C:\\proba", "agenda.txt");
```

- Comprobamos se existe o ficheiro con **exists()** e se é así o borramos con:

```
ficheiro.delete();
```

Para crear directorios, poderíamos facer:

```
import java.io.File;

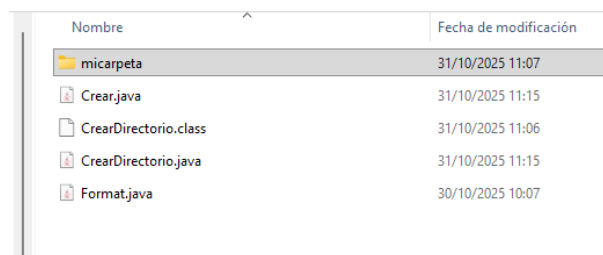
public class CrearDirectorio {

    public static void main(String[] args) {
        try {
            // Declaración de variables
            String directorio = "micarpeta";
            //Ver directorio de traballo
            System.out.println("Directorio de traballo: " +
System.getProperty("user.dir"));

            // Crear un subdirectorio micarpeta dentro del directorio donde
            //está el archivo .java
            boolean exito = ((new File(directorio)).mkdir());
            if (exito) {
                System.out.println("Directorio: " + directorio + "
creado");
            }

        } catch (Exception e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Veriamos como se crea un subdirectorio “*micarpeta*” dentro do noso directorio de traballo (onde está o **.java**).



Nombre	Fecha de modificación
micarpeta	31/10/2025 11:07
Crear.java	31/10/2025 11:15
CrearDirectorio.class	31/10/2025 11:06
CrearDirectorio.java	31/10/2025 11:15
Format.java	30/10/2025 10:07

Nota: Isto funciona perfectamente se compilamos e executamos desde o terminal. Desde o Visual Studio Code é posible que non funcione. O problema non é do noso código. É posible que se o compilamos e executamos desde un IDE non nos cre o directorio onde nós esperamos. Iso é porque Java interpreta esa ruta relativa ao directorio de traballo (**working directory**) no que se está executando o programa, non necesariamente onde está o arquivo .java.

Por defecto, cando executamos desde un IDE, o directorio de traballo adoita ser: A carpeta do proxecto ou a carpeta onde executamos o comando java no terminal.

Podemos ver cal é ese directorio coa liña

```
System.out.println(System.getProperty("user.dir"));
```

Por iso, é posible que o directorio datos non apareza xunto ao .java, senón nese directorio de traballo.

Podemos optar por:

- Executalo desde terminal.
- Utilizar rutas absolutas.
- Crear un proxecto en lugar de traballar con ficheiros soltos.

Para borrar un directorio con Java temos que borrar cada un dos ficheiros e directorios que este conteña. Ao poder almacenar outros directorios, poderíase percorrer recursivamente o directorio para ir borrando todos os ficheiros.

Pódese listar o contido do directorio con:

```
File[] ficheiros = directorio.listFiles();
```

e entón poder ir borrando. Se o elemento é un directorio, sabémolo mediante o método **isDirectory**.

Este exemplo mostra a lista de ficheiros no directorio actual. O método usado é **list()**, que devolve un array de **String** cos nomes dos ficheiros e directorios contidos no directorio asociado ao obxecto **File**. Para indicar que estamos no directorio actual, creamos un obxecto **File** e pasamos a variable dir co valor ".". Defínese un segundo obxecto de ficheiro usando outro constructor para saber se o ficheiro obtido é un ficheiro ou un directorio:

```
import java.io.*;

public class VerDir {
    public static void main(String[] args) {
        String dir = ".";
        File f = new File(dir); //Creamos o File que representa ao directorio
        String[] archivos = f.list(); //Listase o directorio
        System.out.printf("Ficheros en el directorio actual: %d %n",
archivos.length);
        for (int i = 0; i < archivos.length; i++) {
            File f2 = new File(f, archivos[i]); //Créase un File por cada
ficheiro ou directorio interior
            System.out.printf("Nombre: %s, es fichero?: %b, es directorio?: %b
%n", archivos[i], f2.isFile(),
                                f2.isDirectory());
        }
    }
}
```

Este outro exemplo permite ver información acerca do ficheiro VerInf.java

```
import java.io.*;

public class VerInf {
    public static void main(String[] args) {
        System.out.println("INFORMACIÓN SOBRE O FICHEIRO:");
    }
}
```

```

File f = new File("D:\\00. Curso\\2025-2026\\Programación\\codigo\\UD7\\
VerInf.java");
if(f.exists()){
    System.out.println("Nombre del fichero : "+f.getName());
    System.out.println("Ruta : "+f.getPath());
    System.out.println("Ruta absoluta : "+f.getAbsolutePath());
    System.out.println("Se puede leer : "+f.canRead());
    System.out.println("Se puede escribir : "+f.canWrite());
    System.out.println("Tamaño : "+f.length());
    System.out.println("Es un directorio : "+f.isDirectory());
    System.out.println("Es un fichero : "+f.isFile());
    System.out.println("Nombre del directorio padre: "+f.getParent());
}
}
}

```

Obténdose unha saída similar á seguinte:

```

PS C:\Users\Ana> & 'C:\Program Files\Java\jdk-17.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages'
INFORMACIÓN SOBRE O FICHEIRO:
Nombre del fichero : VerInf.java
Ruta : D:\00. Curso\2025-2026\Programación\codigo\UD7\VerInf.java
Ruta absoluta : D:\00. Curso\2025-2026\Programación\codigo\UD7\VerInf.java
Se puede leer : true
Se puede escribir : true
Tamaño : 877
Es un directorio : false
Es un fichero : true
Nombre del directorio padre: D:\00. Curso\2025-2026\Programación\codigo\UD7
PS C:\Users\Ana>

```

Neste outro exemplo crease un directorio (chamado NUEVODIR) no directorio actual, a continuación créanse dous ficheiros baleiros nese directorio e un deles renoméao. Neste caso para crear os ficheiros defínense 2 parámetros no obxecto File: File(File directorio, String nombrefich), no primeiro indicamos o directorio onde se creará o ficheiro e no segundo indicamos o nome do ficheiro. Posteriormente borramos o obxecto f2:

```

import java.io.*;
public class Crear {
    public static void main(String[] args) {
        File d = new File("NUEVODIR"); //directorio que creo a partir do actual
        File f1 = new File(d,"FICHERO1.TXT");
        File f2 = new File(d,"FICHERO2.TXT");

        d.mkdir();//CREAR DIRECTORIO

        try {
            if (f1.createNewFile())
                System.out.println("FICHEIRO1 creado correctamente...");
            else
                System.out.println("Non se puido crear FICHERO1...");

            if (f2.createNewFile())
                System.out.println("FICHEIRO2 creado correctamente...");

```

```

else
    System.out.println("Non se puido crear FICHERO2...");
} catch (IOException ioe) {ioe.printStackTrace();}

f1.renameTo(new File(d, "FICHERO1NUEVO")); //renombro FICHERO1

try {
    File f3 = new File("NUEVODIR/FICHERO3.TXT");
    f3.createNewFile(); //crea FICHERO3 en NUEVODIR
} catch (IOException ioe) {ioe.printStackTrace();}
if (f2.delete())
    System.out.println("Ficheiro borrado ... ");
else
    System.out.println("Non se puido borrar o ficheiro .. ");
}
}

```

O método `createNewFile()` pode lanzar unha excepción *IOException*, por iso se emprega o bloque `try-catch`.

3.3 Interface FilenameFilter

En ocasións interésanos ver a lista dos arquivos que encaixan cun determinado criterio.

Así, pódenos interesar un filtro para ver os ficheiros modificados despois dunha data, ou os que teñen un tamaño maior do que indiquemos, etc.

O interface **FilenameFilter** pódese usar para crear filtros que establezan criterios de filtrado relativos ao nome dos ficheiros. Unha clase que o implemente debe definir e implementar o método:

```
boolean accept(File dir, String nome)
```

Este método devolverá verdadeiro (true), no caso de que o ficheiro cuxo nome se indica no parámetro nome apareza na lista dos ficheiros do directorio indicado polo parámetro dir.

No seguinte exemplo vemos como se listan os ficheiros do cartafol `c:\datos` que teñan a extensión `.odt`. Usamos `try` e `catch` para capturar as posibles excepcións, como que non exista dito cartafol.

Nombre	Fecha de modificación	Tipo	Tamaño
Ejemplos	03/11/2025 9:20	Carpeta de archivos	
EjerciciosExtra	03/11/2025 9:20	Carpeta de archivos	
Tarefas_UD07_A01_SOL.odt	30/10/2025 12:00	Texto OpenDocu...	37 KB
UD07_A01Fluxos.odt	31/10/2025 9:46	Texto OpenDocu...	261 KB
UD07_A01Fluxos.pdf	30/10/2025 11:29	Documento Adob...	503 KB
UD07_A02Ficheiros.odt	03/11/2025 9:17	Texto OpenDocu...	180 KB

```

import java.io.File;
import java.io.FilenameFilter;

/**
 *
 * @author
 */
public class Filtro implements FilenameFilter {
    String extension;
    Filtro(String extension){
        this.extension=extension;
    }
    @Override
    public boolean accept(File dir, String name){
        return name.endsWith(extension);
    }

    public static void main(String[] args) {
        try {
            File fichero=new File("c:\\datos\\.");
            String[] listadeArchivos;
            listadeArchivos = fichero.list(new Filtro(".odt"));
            int numarchivos = listadeArchivos.length ;

            if (numarchivos < 1)
                System.out.println("No hay archivos que listar");
            else
            {
                for (String listadeArchivo : listadeArchivos) {
                    System.out.println(listadeArchivo);
                }
            }
        }
        catch (Exception ex) {
            System.out.println("Error al buscar en la ruta indicada");
        }
    }
}

```

```
PS C:\Users\Ana> & 'C:\Program Files\Java\jdk1.8.0_291\bin\java.exe'  
Tarefas_UD07_A01_SOL.odt  
UD07_A01Fluxos.odt  
UD07_A02Ficheiros.odt  
PS C:\Users\Ana> |
```

```
}
```

4. Manipulación de ficheiros de texto

A manipulación de ficheiros de texto podería facerse desde as clases de fluxo de bytes **FileInputStream** ou **FileOutputStream**. Porén, cando se trata de traballar especificamente con arquivos de texto é mellor transformar os fluxos anteriores ou ben a **Reader** ou **Writer**, segundo corresponda, mediante as clases **InputStreamReader** e **OutputStreamReader**.

De todos os xeitos, non é recomendable, pois existe unha maneira máis eficiente. A **opción ideal** é traballar directamente coas clases **FileReader** e **FileWriter**, para facer a lectura ou a escritura do ficheiro de texto respectivamente.

Os ficheiros de texto, normalmente son xerados cun editor, almacenan caracteres alfanuméricos nun formato estándar (ASCII, UNICODE, UTF8, etc.) Para traballar con eles usaremos as clases **FileReader** para ler caracteres e **FileWriter** para escribir os caracteres no ficheiro. Cando traballamos con ficheiros, cada vez que lemos ou escribimos debemos facelo dentro dun manexador de excepcións **try-catch**. Ao usar a clase **FileReader** pódese xerar a excepción **FileNotFoundException** (porque o nome do ficheiro non existe ou non é válido) e ao usar a clase **FileWriter** a excepción **IOException** (o disco está cheo ou protexido contra escritura).

4.1 Apertura de ficheiro de texto

A primeira operación que temos que realizar é a de apertura do ficheiro. Neste paso hai que decidir se queremos abrir o ficheiro para só para lectura o tamén para escritura.

Se queremos abrir o ficheiro só para lectura utilizamos a clase **FileReader** que dispón dos seguintes construtores:

```
FileReader(File arquivo);  
FileReader(String nomeArquivo);
```

Como se pode comprobar nas dúas sentenzas anteriores a construción dun obxecto pode facerse ou ben cun parámetro que pode ser de tipo **File** ou un **String**. De ámbolos dous xeitos se pode representar a un determinado arquivo.

Se quixeramos abrir o ficheiro tamén para escritura deberemos utilizar neste caso a clase **FileWriter**. Os construtores dispoñibles son os mesmos que na clase **FileReader**, e permiten abrir o ficheiro directamente para escritura. Esta clase ademais incorpora outros dous construtores cun parámetro extra de tipo booleano, que en caso de valer true, indica

que se abre o arquivo para engadir datos, en caso contrario, abrírase para gravar desde cero co que borrará o contido previo que tivese o arquivo.

```
FileWriter(File arquivo);  
FileWriter (String nomeArquivo);  
FileWriter (File arquivo, boolean engadir);  
FileWriter (String nomeArquivo, boolean engadir);
```

Cando se abre o ficheiro un cursor apunta ao inicio do ficheiro. Este cursor indica a seguinte información do ficheiro á que se pode acceder.

A apertura do fluxo cara ao ficheiro produce unha excepción que hai que controlar. Esta excepción prodúcese cando o ficheiro non existe, cando é un directorio en lugar dun ficheiro ou por calquera outra razón que impida abrir o ficheiro. Esta excepción pode ser de dous tipos:

- **FileNotFoundException** cando se usan os métodos de **FileReader**.
- **IOException** cando se usan os métodos de **FileWriter**.

4.2 Peche de ficheiro de texto

Unha vez que se remate de utilizar o ficheiro debe usarse o método **close** que pecha o fluxo e ademais libera calquera recurso que sexa utilizado polo fluxo. É aconsellable incluír este método dentro do bloque **finally** de forma que sempre se execute.

```
try {  
    FileReader ficheiro = new FileReader("datos.dat")  
    //instrucciones  
} catch (IOException e) {  
    //instrucciones  
} finally {  
    ficheiro.close();  
}
```

Desde a versión 7 de Java pode utilizarse tamén a instrución **try-with-resources**. Esta sentenza declara un ou máis recursos, considerando coma tal un obxecto que debe ser pechado despois de que o programa remate de utilizalo. Desta forma, podémonos asegurar que cada recurso será liberado unha vez que finalice a sentenza.

A continuación, pódese ver un exemplo onde se abre un ficheiro para a súa lectura usando a instrución **try-with-resources**. Unha vez que se executen todas as instrucións dentro do bloque try pecharase o fluxo declarado ao seu comezo.

Exemplo de uso try-with-resources

```
try (FileReader ficheiro = new FileReader("datos.dat")) {  
    // instrucciones  
} catch (IOException e) {  
    // instrucciones  
}
```

4.3 Lectura de ficheiros de texto

Para ler datos dun ficheiro pódese utilizar o método `read()` da clase **FileReader**. Este método permite dúas formas de lectura:

- **Lectura carácter a carácter:** O método `read()` devolve un valor enteiro que representa o carácter lido. Se non quedan máis caracteres por ler, devolve -1, indicando o final do ficheiro.
- **Lectura mediante un array de caracteres:** Tamén existe unha versión do método que recibe un array (`read(char[])`). Neste caso, gárdanse varios caracteres no array e o método devolve o número de caracteres lidos, ou -1 se se chegou ao final.

Cada vez que se chama ao método `read()`, o cursor do ficheiro avanza á seguinte posición. Se se le carácter a carácter, avanza un carácter; se se le usando un array, avanza tantos caracteres como se len.

Este método considérase pouco eficiente para ficheiros de texto, xa que realiza lecturas moi básicas. Por iso, é recomendable empregar a clase **BufferedReader**, que permite unha lectura máis eficiente e cómoda, por exemplo mediante o método `readLine()`, que le liñas completas.

Os principais métodos de **BufferedReader** son:

Método	Descrición
<code>int read()</code>	Le un carácter de forma individual e o devolve coma un enteiro que estará dentro do rango 0...65535. Devolverá -1 cando chegue ao final do fluxo.
<code>int read(char[] buffer, int off, int lonxitude)</code>	Almacena nun array de caracteres o que se le. Débese indicar unha posición desde a que comezará a lectura e o número máximo de caracteres a ler. Devolve un enteiro que será o número de caracteres lidos ou -1 cando chegue ao final do fluxo. Devolverá null cando se alcance o final do fluxo.
<code>String readLine()</code>	Le unha liña completa de texto. Considerarase que a liña rematou cando se atope un salto de liña '\n', un retorno de carro '\r' ou un retorno de carro seguido inmediatamente por un salto de liña.

Estes métodos lanzan unha excepción de tipo **IOException**, que se deberá ter en conta á hora de utilizalos.

Cada vez que se executa o método `read()` o cursor avanza á seguinte posición. En caso de estar lendo de carácter en carácter o cursor avanzará ao seguinte carácter, se se le a liña completa entón o cursor avanzará á seguinte liña.

No seguinte exemplo vemos como ler o contido dun ficheiro carácter a carácter con **FileReader**. Para iso consideremos que no directorio do proxecto temos un ficheiro chamado **datos.dat** co seguinte contido:

```
4    6
5    6
7    1
43   5
65   77
```

```
import java.io.FileReader;
import java.io.FileNotFoundException;
import java.io.IOException;
```

```

public class ExemploRead {
    public static void main(String[] args) {
        String contido = "";
        int dato;
        try {
            // Apertura do fluxo de lectura cara ao ficheiro
            // O cursor sitúase ao comezo do ficheiro
            FileReader ficheiro = new FileReader("datos.dat");
            // Lectura do primeiro carácter
            // O cursor móvese cara ao seguinte carácter
            dato = ficheiro.read();
            // Mentres o valor que retorne o método de lectura non sexa -1
            // significa que aínda hai máis caracteres por ler no ficheiro
            while (dato != -1) {
                // O valor do método de lectura devolve un enteiro que debe
                // ser convertido correctamente para a súa visualización
                contido = contido + (char) dato;
                dato = ficheiro.read();
            }
            System.out.println("O contido do ficheiro é o seguinte:\n" +
contido);

            // Peche do fluxo e liberación d e recursos
            ficheiro.close();
            // Pode evitarse o primeiro catch, posto que
FileNotFoundException é unha
            // excepción filla de IOException. Desta maneira podemos
diferenciar os erros
        } catch (FileNotFoundException e) {
            System.out.println("ERRO: O ficheiro é un directorio ou ben non
existe.");
        } catch (IOException e) {
            System.out.println("ERRO: Ocorreu un erro na lectura do
ficheiro");
        }
    }
}

```

```

PS H:\00. Curso\2025-2026\Programación\codigo\ud7\ExemploRead> javac -encoding UTF-8 ExemploRead.java
PS H:\00. Curso\2025-2026\Programación\codigo\ud7\ExemploRead> java ExemploRead
O contido do ficheiro é o seguinte:
4      6
5      6
7      1
43     5
65     77

```

Nota: Compilamos coa opción *-encoding UTF-8* para que non se vexan caracteres extraños na terminal.

FileReader non contén métodos que nos permiten ler liñas completas, pero **BufferedReader** si; ten o método *readLine()* que le unha liña do ficheiro e devólvea, ou devolve null se non hai nada para ler ou chegamos ao final do ficheiro. Tamén ten o método *read()*

para ler un carácter. Para construír un **BufferedReader** necesitamos a clase **FileReader**:

```
BufferedReader fichero = new BufferedReader(new FileReader(NombreFichero));
```

O seguinte exemplo le o ficheiro **Ficheiro.txt** liña por liña e móstraas por pantalla, neste caso, as instrucións foron agrupadas dentro dun bloque de try-catch:

```
import java.io.*;

public class LeerFichTextoBuf {

    public static void main(String[] args) {
        try{
            File fic = new File("Ficheiro.txt");//declara fichero
            BufferedReader fichero = new BufferedReader(
                new FileReader(fic));

            String linea;
            while((linea = fichero.readLine())!=null)
                System.out.println(linea);
            fichero.close();
        }
        catch (FileNotFoundException fn ){
            System.out.println("No se encuentra el fichero");}
        catch (IOException io) {
            System.out.println("Error de E/S ");}

    }
}
```

```
PS H:\00. Curso\2025-2026\Programación\codigo\ud7\LeerFichTextoBuf> javac -encoding UTF-8 LeerFichTextoBuf.java
PS H:\00. Curso\2025-2026\Programación\codigo\ud7\LeerFichTextoBuf> java LeerFichTextoBuf
Este Ão un ficheiro
con varias liÃas
para ler
desde Java.
PS H:\00. Curso\2025-2026\Programación\codigo\ud7\LeerFichTextoBuf>
```

FileReader non permite cambiar a codificación.

Outra opción sería, en lugar de **FileReader**, usar un **FileInputStream** dentro dun **InputStreamReader**:

```
import java.io.*;

public class ExemploReadLine {

    public static void main(String[] args) {
        try {
            File fic = new File("Ficheiro.txt"); // declara fichero
            BufferedReader fichero = new BufferedReader(
                new InputStreamReader(new FileInputStream(fic), "UTF-8")
            );

            String linea;
```

```

        while ((linea = fichero.readLine()) != null)
            System.out.println(linea);

        fichero.close();
    }
    catch (FileNotFoundException fn) {
        System.out.println("No se encuentra el fichero");
    }
    catch (IOException io) {
        System.out.println("Error de E/S");
    }
}
}

```

```

PS H:\00. Curso\2025-2026\Programación\codigo\ud7\LeerFichTextoBuf> java EjemploReadLine
Este é un ficheiro
con varias liñas
para ler
desde Java.
PS H:\00. Curso\2025-2026\Programación\codigo\ud7\LeerFichTextoBuf>

```

4.4 Escritura en ficheiros de texto

Para poder escribir no ficheiro con **FileWriter**, previamente teremos que abrílo para escritura como se viu anteriormente. Unha vez que se pode escribir no ficheiro podemos utilizar o método *write()*, que permite escribir tanto un único carácter, coma un String ou un array de bytes.

Tamén temos o método *append(char c)*, que permite engadir un carácter a un ficheiro.

```

import java.io.*;

public class EscribirFichTextoFW {
    public static void main(String[] args) throws IOException {
        File fichero = new File("FichTexto.txt");//declara ficheiro
        FileWriter fic = new FileWriter(fichero); //crear o fluxo de saída
        String cadena ="Esto é unha proba con FileWriter";
        char[] cad = cadena.toCharArray();//converte un String en array de
        caracteres

        for(int i=0; i<cad.length; i++)
            fic.write(cad[i]); //Vaise escribindo un carácter

        fic.append('*'); //engado ao final un *
        fic.write(cad);//escribir un array de caracteres
        String c="\n*esto é o ultimo*";
        fic.write(c);//escribir un String

        String prov[] = {"Albacete","Avila","Badajoz",
            "Cáceres","Huelva","Jaén",
            "Madrid","Segovia","Soria","Toledo",

```

```

        "Valladolid", "Zamora"};

    fic.write("\n");
    for(int i=0; i<prov.length; i++) {
        fic.write(prov[i]);
        fic.write("\n");
    }

    fic.close();    //pechar fichero

}
}

```

Ao igual que ocorría na lectura de ficheiros, se utilizamos un buffer para escribir en ficheiros, coa clase **BufferedWriter**, podemos utilizar ademais o método **newLine()** que vai escribir un salto de liña no arquivo. Co uso deste método evitamos o problema da compatibilidade entre plataformas, xa que os caracteres de cambio de parágrafo son distintos segundo cada sistema operativo.

```
BufferedWriter fichero = new BufferedWriter (new FileWriter (FileName));
```

O seguinte exemplo escribe 10 filas de caracteres nun ficheiro de texto e despois de escribir cada fila salta unha liña co método **newLine()**:

```

import java.io.*;

public class EscribirFichTextoBuf {
    public static void main(String[] args) {
        try {
            BufferedWriter fichero = new BufferedWriter(new
FileWriter("FichTexto1.txt"));
            for (int i = 1; i < 11; i++) {
                fichero.write("Fila numero: " + i); // escribe una liña
                fichero.newLine(); // escribe un salto de liña
            }
            fichero.close();
        } catch (FileNotFoundException fn) {
            System.out.println("No se encuentra el fichero");
        } catch (IOException io) {
            System.out.println("Error de E/S ");
        }
    }
}

```

Se ademais non queremos que existan problemas de codificación, en lugar de meter un **FileWriter** podemos meter un **FileOutputStream** dentro dun **OutputStreamWriter**:

```

import java.io.*;

public class EscribirFichTextoFOS {
    public static void main(String[] args) throws IOException {
        File fichero = new File("FichTexto2.txt"); // declara ficheiro

        // Crear fluxo de saída con UTF-8
        BufferedWriter fic = new BufferedWriter(
            new OutputStreamWriter(new FileOutputStream(fichero), "UTF-
8"));

        String cadena = "Esto é unha proba con FileWriter";
        char[] cad = cadena.toCharArray(); // converte un String en array
de caracteres

        for (int i = 0; i < cad.length; i++)
            fic.write(cad[i]); // escribe cada carácter

        fic.append('*'); // engadir un *
        fic.write(cad); // escribir array de caracteres

        String c = "\n*esto é o ultimo*";
        fic.write(c); // escribir un String

        String prov[] = { "Albacete", "Avila", "Badajoz",
            "Cáceres", "Huelva", "Jaén",
            "Madrid", "Segovia", "Soria", "Toledo",
            "Valladolid", "Zamora" };

        fic.write("\n");
        for (int i = 0; i < prov.length; i++) {
            fic.write(prov[i]);
            fic.write("\n");
        }

        fic.close(); // pechar ficheiro
    }
}

```

A clase **PrintWriter**, que también deriva de **Writer**, posee os métodos *print(String)* e *println(String)* (idénticos aos de *System.out*) para escribir nun ficheiro. Ambos reciben un **String** e escribeno nun ficheiro, o segundo método, ademáis , produce un salto de liña. Para construír un **PrintWriter** necesitamos a clase **FileWriter**:

```

PrintWriter fichero = new PrintWriter(new FileWriter(NombreFichero));

```

O exemplo anterior empregando a clase **PrintWriter** e o método *println()* quedaría así:

```

import java.io.*;

```

```

public class EscribirFichTextoPW {
    public static void main(String[] args) throws IOException {
        File fichero = new File("FichTexto3.txt");

        // PrintWriter con FileWriter
        PrintWriter pw = new PrintWriter(new FileWriter(fichero));

        String cadena = "Esto é unha proba con PrintWriter y FileWriter";

        pw.print(cadena); // escribir string
        pw.println('*'); // engadir *
        pw.println(cadena);

        String prov[] = { "Albacete", "Avila", "Badajoz",
                          "Cáceres", "Huelva", "Jaén",
                          "Madrid", "Segovia", "Soria", "Toledo",
                          "Valladolid", "Zamora" };

        for (String p : prov) {
            pw.println(p);
        }

        pw.close();
    }
}

```

5. Manipulación de ficheiros binarios

Un ficheiro binario está formado por unha secuencia de **bytes**. Teñen a vantaxe de que ocupan menos espazo no disco. Dependendo de se coñecemos a estrutura interna do ficheiro podemos ler o seu contido dunha forma ou doutra. Se non coñecemos a estrutura entón debemos ler o ficheiro byte a byte. En caso de coñecer a estrutura e de saber o tipo (int, float, char, etc) e a orde dos datos podemos utilizar métodos máis específicos para a lectura de cada tipo de dato.

5.1 Apertura de ficheiros binarios

A clase que nos permite abrir un ficheiro binario para a súa lectura é **FileInputStream** e ten os seguintes construtores:

```

FileInputStream(String ruta)
FileInputStream(File objetoFile);

```

Os métodos que a clase **FileInputStream** para a lectura de bytes devolven o número de bytes lidos ou -1 se chegou ao final do ficheiro:

Método	Función
<code>int read ()</code>	Le un único byte e devólveo. Devolve -1 se xa non hai máis datos.
<code>int read (byte [] b)</code>	Intenta ler ata <code>b.length</code> bytes. Garda os datos no array <code>b</code> . Devolve o número de bytes lidos (pode ser menor que o tamaño do array) ou -1 se está ao final do ficheiro
<code>int read (byte[] b, int desprazamento, int n)</code>	Le ata <code>n</code> bytes. Comeza a escribir no array <code>b</code> desde a posición <code>desprazamento</code> . Devolve o número de bytes lidos ou -1 se non hai máis datos

A clase que nos permite abrir un ficheiro binario para a súa escritura é **FileOutputStream** e ten os seguintes construtores:

```
FileOutputStream(String ruta)
FileOutputStream(File objetoFile);
FileOutputStream(String ruta, boolean append)
FileOutputStream(File objetoFile, boolean append)
```

Son idénticos que no caso dos ficheiros de texto, polo que as dúas primeiras opcións abren o ficheiro para escritura de maneira que os datos existentes se perderán. Se se empregan as últimas dúas opcións e se o parámetro **booleano** está a *true*, entón os datos anteriores permanecen e os novos **engadiranse** ao final do ficheiro, en caso de que sexa *false*, borraranse os datos existentes antes de proceder coa escritura.

Todos os construtores mencionados neste apartado, lanzan unha excepción **FileNotFoundException** cando o ficheiro non existe ou se non se puido crear o ficheiro en caso de abrir o ficheiro para escritura.

Os métodos que proporciona a clase **FileOutputStream** para escritura de bytes son:

Método	Función
<code>void write(int b)</code>	Escribe un único byte. Aínda que o parámetro é un int, só usa os 8 bits menos significativos (un byte)
<code>void write (byte [] b)</code>	Escribe todos os bytes do array <code>b</code> . Equivale a escribir desde <code>b[0]</code> ata <code>b[b.length - 1]</code> .
<code>void write (byte[] b , int desprazamento, int n)</code>	Escribe <code>n</code> bytes comezando desde a posición <code>desprazamento</code> do array <code>b</code>

5.2 Peche de ficheiros binarios

Ao igual que ocorre cos ficheiros de texto, os fluxos de datos binarios deben ser pechados para liberar calquera recurso que estivese utilizando. Para iso, existe o mesmo método que no caso de ficheiros de texto:

```
close () ;
```

Se hai un erro de entrada/saída, este método lanza unha excepción de tipo **IOException**.

É aconsellable incluír esta instrución de peche dentro do bloque **finally** para que sempre se execute.

Pode evitarse ter que estar pendente do peche do fluxo declarando o fluxo nunha instrución **try-with-resources** para que sexa o propio programa o que se encargue do peche do fluxo ao final da execución da instrución.

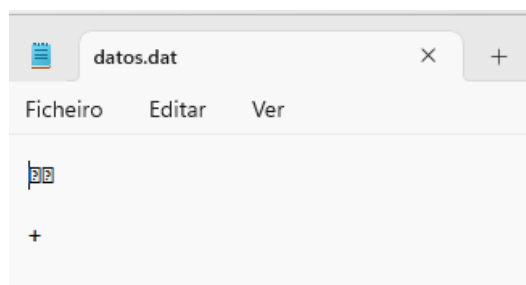
5.3 Escritura en ficheiros binarios

A clase utilizada para abrir un fluxo de bytes en modo escritura, **FileOutputStream**, proporciona o método **write()** para escribir bytes no ficheiro. Cando usemos este método hai que ter en conta que lanza unha excepción de tipo **IOException**.

```
import java.io.FileOutputStream;
import java.io.IOException;

public class EscrituraBinario {
    public static void main(String[] args) {
        FileOutputStream ficheiro = null;
        try {
            ficheiro = new FileOutputStream("datos.dat");
            byte[] datos = { 5, 6, 7, 12, 12, 43 };
            ficheiro.write(datos);
        } catch (IOException e) {
            System.out.println("ERRO: Ocorreu un erro na escritura do
ficheiro");
        } finally {
            try {
                // Comprobamos se o fluxo está aberto.
                // Se non está aberto non é necesario pechalo
                if (ficheiro != null) {
                    ficheiro.close();
                }
            } catch (IOException e) {
                System.out.println("ERRO: Ocorreu un erro no peche do
ficheiro");
            }
        }
    }
}
```

O resultado da execución do código anterior é a creación do arquivo datos.dat coa seguinte información, no lexible para o usuario:



Para ler o seu contido debemos crear un programa que lea do ficheiro e mostre o seu contido.

Para engadir bytes ao final do ficheiro usaremos **FileOutputStream** da seguinte forma, colocando no segundo parámetro do constructor o valor `true`:

```
FileOutputStream fileout = new FileOutputStream(ficheiro, true);
```

Se quixeramos escribir no ficheiro datos de tipo primitivo, como por exemplo datos de tipo **int**, podemos crear un obxecto **DataOutputStream** a partir do obxecto **FileOutputStream**, xa que esta clase proporciona métodos específicos para a escritura de datos de tipo primitivo no ficheiro.

Para crear un obxecto de tipo **DataOutputStream** utilízase o seguinte construtor:

```
DataOutputStream(OutputStream nome);
```

De tal maneira que para abrir un obxecto **DataOutputStream** faremos:

```
File fichero = new File ("FichData.dat");
FileOutputStream fileout = new FileOutputStream(fichero );
DataOutputStream dataOS = new DataOutputStream(fileout );
```

ou

```
File fichero = new File (" FichData.dat" );
DataOutputStream dataOS = new DataOutputStream (new
FileOutputStream(fichero));
```

Os métodos para escribir os datos de tipo primitivo teñen a seguinte forma:

```
writeXxx(); //Onde Xxx é o nome do tipo primitivo.
```

Ao igual que o método xeral para escribir bytes, estes métodos tamén lanzan a excepción **IOException**.

No seguinte exemplo imos insertar datos no ficheiro `FichData.dat`, os datos tómaos de dous arrays, un contén os nomes dunha serie de persoas e outro as súas idades, percorremos e imos escribindo no ficheiro o nome (mediante o método **writeUTF(String)**) e a idade (mediante o método **writeInt(int)**):

```
import java.io.*;
public class EscribirFichData {
    public static void main(String[] args) throws IOException {

        File fichero = new File("FichData.dat");
        DataOutputStream dataOS = new
            DataOutputStream(new FileOutputStream(fichero));

        String nombres[] = {"Ana", "Luis
Miguel", "Alicia", "Pedro", "Manuel", "Andrés",
```

```

        "Julio","Antonio","María Jesús"};

int edades[] = {14,15,13,15,16,12,16,14,13};

for (int i=0;i<edades.length; i++){
    dataOS.writeUTF(nombres[i]); //inserta nome
    dataOS.writeInt(edades[i]); //inserta idade
}
dataOS.close(); //pechar stream
}
}

```

Os distintos métodos tanto para lectura como para escritura de tipos concretos en ficheiros de bytes son os seguintes:

Tipo de dato	Método de escritura (DataOutputStream)	Método de lectura (DataInputStream)
byte	writeByte(int v)	readByte()
boolean	writeBoolean(boolean v)	readBoolean()
char	writeChar(int v)	readChar()
short	writeShort(int v)	readShort()
int	writeInt(int v)	readInt()
long	writeLong(long v)	readLong()
float	writeFloat(float v)	readFloat()
double	writeDouble(double v)	readDouble()
cadea (String)	writeUTF(String s)	readUTF()

5.4 Lectura de ficheiros binarios

A clase utilizada para abrir un fluxo de bytes en modo lectura, **FileInputStream**, proporciona o método **read()** para ler bytes desde o ficheiro. Cando usemos este método hai que ter en conta que lanza unha excepción de tipo **IOException**.

Método	Función
int read ()	Le un byte e devólveo ou -1 se chega ao final
int read (byte [] b)	Le ata b.length bytes de datos dunha matriz de bytes. Devolve número de bytes lidos ou -1 (EOF)
int read (byte[] b, int desprazamento, int n)	Le ata n bytes da matriz b comezando por b[desprazamento] e devolve o número lido de bytes ou -1 (EOF)
int available()	Obtén o número estimado de bytes que se poden ler do fluxo sen bloquear. Adoita coincidir co que queda, pero non se debe usar para controlar bucles de lectura

Se quixeramos ler do ficheiro datos de tipo primitivo, como por exemplo datos de tipo int, podemos crear un obxecto **DataInputStream** a partir do obxecto **FileInputStream**, xa que esta clase proporciona métodos específicos para a lectura de datos de tipo primitivo no ficheiro.

No seguinte exemplo pode comprobarse como se faría a lectura do ficheiro datos.dat creado no apartado anterior:

```

import java.io.IOException;
import java.io.FileInputStream;

public class LecturaBinario {
    public static void main(String[] args) {

        FileInputStream fichero = null;
        try {
            fichero = new FileInputStream("datos.dat");
            byte[] datos = new byte[fichero.available()]; // le o número de
bytes que existen
            int valor = fichero.read(datos);
            if (valor != 0) {
                for (int i = 0; i < valor; i++) {
                    System.out.print(datos[i] + "\t");
                }
            }
        } catch (IOException e) {
            System.out.println("ERRO: Ocorreu un erro na escritura do
fichero");
        } finally {
            try {
                // Comprobamos se o fluxo está aberto.
                // Se non está aberto non é necesario pechalo
                if (fichero != null) {
                    fichero.close();
                }
            } catch (IOException e) {
                System.out.println("ERRO: Ocorreu un erro no peche do
fichero");
            }
        }
    }
}

```

E aínda que abrindo o ficheiro co bloc de notas, o contido non sexa accesible, si é capaz de ler o seu contido.

```

PS H:\00. Curso\2025-2026\Programación\codigo\UD7> java LecturaBinario
5      6      7      12      12      43
PS H:\00. Curso\2025-2026\Programación\codigo\UD7>

```

Para crear un obxecto de tipo **DataInputStream** utilízase o seguinte construtor:

```
DataInputStream(InputStream nome);
```

Os métodos para ler os datos de tipo primitivo teñen a seguinte forma:

```
readXxx(); //Onde Xxx é o nome do tipo primitivo.
```

Ao igual que o método xeral para ler bytes, estes métodos tamén poden lanzar a excepción **IOException**.

Cando un método **readXxx()** alcanza o final do ficheiro lanza unha excepción **EOFException**.

Imos ler o ficheiro FichData.dat creado no apartado anterior:

```
import java.io.*;
public class LeerFichData {
    public static void main(String[] args) throws IOException {
        File fichero = new File("FichData.dat");
        DataInputStream dataIS = new
            DataInputStream(new FileInputStream(fichero));

        String n;
        int e;

        try {
            while (true) {
                n = dataIS.readUTF(); //recupera o nome
                e = dataIS.readInt(); //recupera a idade
                System.out.println("Nome: " + n +
                    ", idade: " + e);
            }
        } catch (EOFException eo) {}

        dataIS.close(); //pechar stream
    }
}
```

```
PS H:\00. Curso\2025-2026\Programación\codigo\UD7> java LeerFichData
Nome: Ana, idade: 14
Nome: Luis Miguel, idade: 15
Nome: Alicia, idade: 13
Nome: Pedro, idade: 15
Nome: Manuel, idade: 16
Nome: Andrés, idade: 12
Nome: Julio, idade: 16
Nome: Antonio, idade: 14
Nome: María Jesús, idade: 13
PS H:\00. Curso\2025-2026\Programación\codigo\UD7>
```

6. Ficheiros de acceso aleatorio

Os tipos de ficheiros vistos ata agora son secuenciais, o que quere dicir que para acceder a un determinado rexistro débese percorrer secuencialmente desde o inicio do ficheiro e pasar por todos os rexistros ata atopar o rexistro que nos interese.

Agora imos a falar de como traballar cos ficheiros de acceso aleatorio. Este tipo de ficheiros teñen as seguintes características:

- Non se trata dun fluxo de datos como nos ficheiros de texto ou binarios.
- Almacena un conxunto de bytes.
- Permite o acceso aleatorio a un determinado rexistro, desta forma non é necesario percorrer os rexistros anteriores para acceder a un rexistro concreto.
- Pódese abrir para lectura ou para lectura e escritura.
- Ten un cursor ou índice chamado **file pointer** (nos usaremos o termo **punteiro**), que sinala cara un punto do ficheiro. Inicialmente apunta ao comezo do ficheiro e pódese mover á outra posición diferente. As operacións de lectura comezan a ler bytes a partir da posición á que apunta o punteiro nese instante e o desprazan tantos bytes como foron lidos. No caso das operacións de escritura, os bytes empézanse a escribir a partir da posición indicada polo punteiro e este avanza ata pasados os bytes que foron escritos.
- Dispón de operacións de lectura e escritura baseadas no tipo de dato a ler ou a escribir no ficheiro.

A clase que permite o uso de ficheiros de acceso aleatorio é a clase **RandomAccessFile** que tamén se atopa no paquete **java.io**. Esta clase proporciona dous construtores:

```
RandomAccessFile(File arquivo, String modo);  
RandomAccessFile(String nome, String modo);
```

Os dous construtores poden lanzar a excepción **FileNotFoundException** que hai que tratar.

A forma na que indicamos o arquivo que queremos abrir en modo de acceso aleatorio o podemos facer creando un obxecto de tipo **File** ou ben indicando a cadea de texto que corresponde co ficheiro.

Ambos construtores teñen un parámetro obrigatorio que indica o modo no que se abrirá o ficheiro. Os valores permitidos para este parámetro son as que se indican na seguinte táboa:

Modo	Descrición
"r"	Abre o ficheiro para só lectura. Se utilizamos algún dos métodos de escritura sobre o obxecto creado lanzarase unha excepción IOException .
"rw"	Abre o ficheiro para lectura e escritura. Se o ficheiro non existe entón intentará crealo.
"rws"	Abre o ficheiro para lectura. Cada cambio no contido e nos metadatos do ficheiro escríbese de forma sincronizada no disco (non queda só na caché).
"rwd"	Abre o ficheiro para lectura e escritura. Cada cambio no contido do ficheiro escríbese de forma sincronizada no disco, pero os metadatos poden non gardarse inmediatamente.

Os métodos xerais de lectura e escritura que proporciona a clase son similares aos vistos na clase **DataInputStream**. Ao igual que nesta clase teñen a seguinte forma:

```
readXxx(); //Métodos de lectura onde Xxx é o nome do tipo primitivo.  
writeXxx(); //Métodos de escritura onde Xxx é o nome do tipo primitivo.
```

Ao igual que ocurría cos ficheiros binarios, todos os métodos de lectura lanzan a excepción **EOFException** cando por mor da lectura se alcanza o final do ficheiro.

Respecto ao funcionamento dos métodos de escritura, hai que aclarar, que cando se utiliza un método **write** polo medio do ficheiro, é dicir, cando a posición do punteiro non estea ao final do ficheiro, entón o método o que vai a facer é sobrescribir a información xa existente no arquivo, non a engade. Pola contra, se a posición do punteiro está ao final do arquivo, se utilizamos un método de escritura, entón si que escribirá ao final do arquivo e incrementará a lonxitude do mesmo.

Hai outros métodos que son máis específicos do uso desta clase que son os que se mostrar na seguinte táboa:

Método	Descrición
long getFilePointer()	Devolve a posición actual do punteiro medida desde o comezo do arquivo , a partir do cal se fará a seguinte lectura ou escritura.
long length()	Devolve a lonxitude do ficheiro. A posición length() marca o final do ficheiro.
void seek(long posicion)	Establece o desprazamento do punteiro, medido desde o comezo do arquivo, a partir do cal se fará a seguinte lectura ou escritura. O desprazamento pode situarse máis alá do final do arquivo, pero isto non fará que se modifique a lonxitude do arquivo. A lonxitude do arquivo só se modificará cando despois dunha escritura, o desprazamento se estableza máis alá do final do arquivo. Devolve unha excepción IOException se a posición indicada é menor que 0 ou se ocorre algún erro de entrada/saída.
long setLength(long lonxitude)	Establece a lonxitude do ficheiro. Pódense dar dúas situacións: - Se a lonxitude é maior que o tamaño dado polo método length(), entón se estenderá o arquivo. - Se a lonxitude é menor que o tamaño dado polo método length(), entón o ficheiro será truncado. Se a onde apunta actualmente o punteiro (valor dado polo método getFilePointer()) é maior que a nova lonxitude entón o modificarase o punteiro para que apunte á nova lonxitude.
int skipBytes (int desplazamiento)	Intenta desprazar o punteiro desde a posición actual o número de bytes indicados en desplazamiento. Non le nin escribe os bytes saltados. Se non hai suficientes bytes dispoñibles ata o final do ficheiro, salta só os que poida. Devolve o número real de bytes saltados (que pode ser menor que n).

Todos os métodos da táboa devolven unha excepción **IOException** se ocorre algún erro de entrada/saída.

A continuación temos un pequeno exemplo de lectura e escritura dun ficheiro de acceso aleatorio.

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class ExemploAleatorio {
    public static void main(String[] args) {
        // Abrimos o ficheiro para lectura e escritura usando try-with-
        resources para
        // que ao final do bloque se peche automaticamente o ficheiro.
        try (RandomAccessFile ficheiro = new
RandomAccessFile("ficheiro.dat", "rw")) {
            // Comprobamos que o punteiro sinala ao comezo do ficheiro unha
            vez que se abre
            System.out.println("Posición actual do punteiro: " +
ficheiro.getFilePointer());
```



```

        // Escribimos un valor de tipo inteiro
        ficheiro.writeInt(20);
        // Escribimos un valor de tipo float
        ficheiro.writeFloat(6.45f);
        // Comprobamos onde está colocado o punteiro despois das dúas
        escrituras
        System.out.println("Posición actual do punteiro: " +
        ficheiro.getFilePointer());
        // Comprobamos a lonxitude do ficheiro despois das dúas
        escrituras
        System.out.println("Tamaño do ficheiro: " + ficheiro.length());
        // Movemos o punteiro ao comezo do ficheiro para poder ler todo
        o seu contido
        ficheiro.seek(0);
        // Lemos os dous valores, primeiro o inteiro e logo o float
        System.out.println("Primeiro valor: " + ficheiro.readInt());
        System.out.println("Primeiro valor: " + ficheiro.readFloat());
    } catch (FileNotFoundException fnfe) {
        System.out.println("ERRO: O arquivo indicado pode non ser un
        ficheiro.");
    } catch (IOException ioe) {
        System.out.println("ERRO: Houbo un erro de entrada/saída.");
    }
}
}

```

Outro exemplo mais completo sería este onde se insiren os datos dos empregados nun ficheiro aleatorio. Os datos para inserir son: apelidos, departamentos e salarios, obtéñense de varios arrays que se enchen no programa, os datos introdúcense secuencialmente polo que non será necesario usar o método `seek()`. Para cada empregado inserírase tamén un identificador (maior que 0) que coincidirá co índice +1 co que se percorren os arrays. A lonxitude do rexistro de cada empregado é o mesmo (36 bytes) e os tipos que se insiren e o seu tamaño en bytes é o seguinte:

- En primeiro lugar insírese un inteiro, que é o identificador, ocupando 4 bytes.
- A continuación unha cadea de 10 caracteres, é o apelido. Como Java usa os caracteres UNICODE, cada carácter dunha cadea de caracteres ocupa 16 bits (2 bytes), polo tanto, o apelido ocupa 20 bytes.
- Un tipo inteiro que é o departamento, ocupa 4 bytes.
- Un tipo Double que é o salario, ocupa 8 bytes.

Tamaño de outros tipos: short (2 bytes), byte (byte 1), long (8 bytes), boolean (1 bit), float (4 bytes), etc.

O ficheiro ábrese no modo "rw" para ler e escribir.

Usamos a estrutura **try-with-resources**. O que vai entre parénteses () despois de try son os recursos que deben pechase automaticamente ao rematar o bloque, é dicir, obxectos que implementan a interface `AutoCloseable` (ou a máis antiga `Closeable`).

```

import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

public class FicheiroEmpregadoInserir {

    static final int TAM_REXISTRO = 36; // tamaño fixo do rexistro

    public static void main(String[] args) throws IOException {
        String[] apelidos = { "Gomez", "Lopez", "Perez", "Vazquez", "Sanchez" };
        int[] departamentos = { 10, 20, 10, 30, 10 };
        double[] salarios = { 1000.5, 1200.0, 1100.75, 1300.0, 1250.25 };

        try (RandomAccessFile file = new RandomAccessFile("empregados.dat",
"rw")) {

            for (int i = 0; i < apelidos.length; i++) { //percorremos os arrays
                int id = i + 1; // identificador = índice + 1
                file.writeInt(id); // 4 bytes

                // Apelido de lonxitude fixa (10 caracteres = 20 bytes)
                StringBuffer sb = new StringBuffer(apelidos[i]); //Buffer para
almacenar apelido
                sb.setLength(10); //Se é máis curto énchese con espazos.
//Se é máis largo, cúrtase.
                file.writeChars(sb.toString());
                //Non se pode facer directamente con String porque ten que ter
//un tamaño fixo

                file.writeInt(departamentos[i]); // 4 bytes
                file.writeDouble(salarios[i]); // 8 bytes
            }

            System.out.println("Ficheiro 'empregados.dat' creado e rexistros
inseridos correctamente.");

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

O seguinte exemplo toma o ficheiro anterior e visualiza todos os rexistros. O posicionamento para empezar a percorrer os rexistros empeza en 0, para recuperar os seguintes rexistros hai que sumar 36 (tamaño do rexistro) á variable empregada para o posicionamento:

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class FicheiroEmpregadoLer {

    static final int TAM_REXISTRO = 36;

    public static void main(String[] args) {
        try (RandomAccessFile file = new RandomAccessFile("empregados.dat",
"r")) {

            long posicion = 0; // posición inicial do primeiro rexistro

            System.out.println("== LISTA DE EMPREGADOS ==");

            // Bucle ata o final do ficheiro
            while (posicion < file.length()) {

                file.seek(posicion); // situarse na posición do rexistro

                int id = file.readInt(); // 4 bytes → ID

                // Ler apelido de 10 caracteres (20 bytes)
                char[] apelidoChars = new char[10];
                for (int i = 0; i < apelidoChars.length; i++) {
                    apelidoChars[i] = file.readChar();
                }
                String apelido = new String(apelidoChars).trim(); //
eliminar espazos sobrantes

                int dep = file.readInt(); // 4 bytes → departamento
                double salario = file.readDouble(); // 8 bytes → salario

                // Mostrar o rexistro
                System.out.printf("ID: %d | Apelido: %-10s | Dep: %d |
Salario: %.2f%n",
                                id, apelido, dep, salario);

                // Pasar ao seguinte rexistro (sumar 36 bytes)
                posicion += TAM_REXISTRO;
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }
}
}

```

A saída é algo así:

```

PS H:\00. Curso\2025-2026\Programación\codigo\UD7> java FicheroEmpleadoLer
== LISTA DE EMPREGADOS ==
ID: 1 | Apellido: Gomez      | Dep: 10 | Salario: 1000,50
ID: 2 | Apellido: Lopez        | Dep: 20 | Salario: 1200,00
ID: 3 | Apellido: Perez         | Dep: 10 | Salario: 1100,75
ID: 4 | Apellido: Vazquez       | Dep: 30 | Salario: 1300,00
ID: 5 | Apellido: Sanchez       | Dep: 10 | Salario: 1250,25
PS H:\00. Curso\2025-2026\Programación\codigo\UD7>

```

Para consultar un empregado específico non é necesario percorrer todos os rexistros do ficheiro, coñecendo o seu identificador podemos acceder á posición que ocupa dentro del e obter os seus datos. Por exemplo, supoñendo que queremos obter os datos do empregado con identificador 5, para calcular a posición debemos ter en conta os bytes ocupados por cada rexistro (neste exemplo son 36 bytes):

```

int identificador = 5;
//calculo donde empieza el registro
posicion = (identificador - 1 ) * 36;
if (posicion >= file.length())
    System.out.printf("ID: %d, NON EXISTE EMPREGADO... " ,
identificador);
else{
    file.seek(posicion); //posicionámosnos
    id = file.readInt(); //obtemos id de empleado
    //obter resto dos datos, como no exemplo anterior
}

```

Para engadir rexistros a partir do último insertado debemos posicionar o punteiro do ficheiro ao final do mesmo:

```

long posicion= file.Length();
file.seek (posicion);

```

Para inserir un novo rexistro aplicamos a función ao identificador para calcular a posición. O seguinte exemplo insire un empregado con identificador 20, hai que calcular a posición onde irá o rexistro dentro do ficheiro (identificador -1) *36 bytes:

```

StringBuffer buffer = null; //buffer para almacenar apellido
String apellido = "RODRIGUEZ"; //apellido a insertar
Double salario = 1530.25; //salario

```

```

int id = 20; //id do empregao
int dep = 10; //dep do empregado

long posicion = (id - 1) * 36; //calculamos a posición

file.seek (posicion); //Posicionámosnos
file.writeInt(id) ; //Escríbese id
buffer = new StringBuffer(apellido);
buffer.setLength(10); //10 caracteres para o apelido
file.writeChars (buffer.toString()); //insertar apelido
file.writeInt(dep); //insertar departamento
file.writeDouble(salario); //insertar salario

file.close(); //pechar ficheiro

```

Para modificar un rexistro determinado, accedemos á súa posición e efectuamos as modificacións. O ficheiro debe abrirse en modo "rw" . Por exemplo, para cambiar o departamento e salario do empleado con identificador 4 escribimos o seguinte:

```

int rexistro = 4; //id a modificar
long posicion = (rexistro-1 ) * 36; //calcula a posición
posicion = posicion + 4 + 20; // sumo o tamaño de ID + apelido
file.seek(posicion); //nos posicionamos
file.writeInt(40); //modifico departamento
file.writeDouble(4000.87); //modifico salario

```

7. Obxectos en ficheiros binarios. Serialización

Se en lugar de querer gardar datos primitivos nun ficheiro queremos almacenar un **obxecto** con varios atributos, como pode ser un traballador (nome, enderezo, posto, departamento...) habería que gardar atributo a atributo, o que sería moi lioso. Java permite gardar obxectos completos en ficheiros binarios se facemos que as clases que definen eses obxectos implementen a interfaz **Serializable**.

A interface **Serializable** non ten métodos nin atributos e serve só para indicar que os obxectos da clase son serializables.

Se queremos enviar un obxecto a través dun fluxo de datos, deberemos convertilo nunha serie de bytes. Isto é o que se coñece como **serialización de obxectos**, que nos permitirá ler e escribir obxectos directamente.

Para ler ou escribir obxectos podemos empregar os obxectos de **ObjectInputStream** e **ObjectOutputStream** que incorporan os métodos **readObject** e **writeObject** respectivamente. Os obxectos que escribamos no fluxo deben ter a capacidade de ser serializables.

- **Object readObject():** empregase para ler un obxecto dun **ObjectInputStream**. Pode lanzar as excepcións **IOException** e **ClassNotFoundException**.

- **void writeObject(Object obj):** emprégase para escribir o obxecto especificado nun **ObjectOutputStream**. Pode lanzar a excepción **IOException**.

A serialización de obxectos de Java permite tomar calquera obxecto que implemente a interface **Serializable** e convertilo nunha secuencia de bits, que pode ser posteriormente restaurada para rexenerar o obxecto orixinal.

No exemplo creamos unha clase **Persona** que implementa a interface **Serializable** e que imos usar para escribir e ler obxectos nun ficheiro binario. A clase ten dous atributos: **nome** e **idade** e os métodos **get** e **set** para obter o valor do atributo e para darlle valor:

```
import java.io.Serializable;

public class Persona implements Serializable{
    private String nombre;
    private int edad;

    public Persona(String nombre,int edad)  {
        this.nombre=nombre;
        this.edad=edad;
    }
    public Persona() {
        this.nombre=null;
    }
    public void setNombre(String nom){nombre=nom;}
    public void setEdad(int ed){edad=ed;}

    public String getNombre(){return nombre;}
    public int getEdad(){return edad;}
} //fin Persona
```

Agora imos escribir obxectos **Persona** nun ficheiro. Necesitamos crear un fluxo de saída a disco con **FileOutputStream** e a continuación crear o fluxo de saída **ObjectOutputStream** que é o que procesa os datos e débese vincular ao ficheiro de **FileOutputStream**:

```
File fichero = new File ( "FichPersona.dat" ) ;
FileOutputStream fileout = new FileOutputStream(fichero) ;
ObjectOutputStream dataOS = new ObjectOutputStream(fileout) ;
```

O método **writeObject()** escribe os obxectos ao fluxo de saída e gárdalos nun ficheiro en disco: **dataOS.writeObject(persona)**.

```
import java.io.*;

public class EscribirFichObject {
    public static void main(String[] args) throws IOException {

        Persona persona; // defino variable persona

        File fichero = new File("FichPersona.dat"); // declara o ficheiro
        FileOutputStream fileout = new FileOutputStream(fichero, true); // crea o fluxo
        de saída
```

```

// conecta el flujo de bytes al flujo de datos
ObjectOutputStream dataOS = new ObjectOutputStream(fileout);

String nombres[] = { "Ana", "Luis Miguel", "Alicia", "Pedro", "Manuel", "Andrés",
    "Julio", "Antonio", "María Jesús" };

int edades[] = { 14, 15, 13, 15, 16, 12, 16, 14, 13 };
System.out.println("GRABO OS DATOS DE PERSOA.");
for (int i = 0; i < edades.length; i++) { // recorro os arrays
    persona = new Persona(nombres[i], edades[i]); // creo a persoa
    dataOS.writeObject(persona); // escribo a persoa no ficheiro
    System.out.println("GRABO OS DATOS DE PERSONA.");
}
dataOS.close(); // pechar stream de saída
}
}

```

Para ler obxectos **Persona** do ficheiro necesitamos o fluxo de entrada a disco **FileInputStream** e a continuación crear o fluxo de entrada **ObjectInputStream** que é o que procesa os datos e debemos vinculalo ao ficheiro de **FileInputStream**:

```

File fichero = new File("FichPersona.dat") ;
FileInputStream filein = new FileInputStream(fichero);
ObjectInputStream dataIS = new ObjectInputStream(filein);

```

O método *readObject()* le os obxectos do fluxo de entrada, pode lanzar a excepción **ClassNotFoundException** e **IOException**, polo que será necesario controlalas. O proceso de lectura faise nun bucle **while(true)**, este encérrase nun bloque **try-catch** xa que a lectura finalizará cando se cheegue ao final do ficheiro, entón, lanzarase a excepción **EOFException**.

```

import java.io.*;

public class LeerFichObject {
    public static void main(String[] args) throws IOException, ClassNotFoundException
    {
        Persona persona; // defino a variable persona
        File fichero = new File("FichPersona.dat");
        ObjectInputStream dataIS = new ObjectInputStream(new
FileInputStream(fichero));

        int i = 1;
        try {
            while (true) { // lectura do ficheiro
                persona = (Persona) dataIS.readObject(); // ler unha Persona
                System.out.print(i + ">");
                i++;
                System.out.printf("Nome: %s, idade: %d %n",
                    persona.getNombre(), persona.getEdad());
            }
        } catch (EOFException e) {
            // Fin do ficheiro
        }
    }
}

```

```

    }
    } catch (EOFException eo) {
        System.out.println("FIN DE LECTURA.");
    } catch (StreamCorruptedException x) {
    }

    dataIS.close(); // pechar stream de entrada
}
}

```

7.1 Problema cos ficheiros de obxectos

Hai un problema cos ficheiros de obxectos. Cando se crea un ficheiro de obxecto, créase un encabezado inicial con información e, a continuación, engádense os obxectos. Se o ficheiro se usa de novo para engadir máis rexistros, créase unha nova cabeceira e engádense os obxectos desde esa cabeceira. O problema xorde ao ler o ficheiro cando na lectura nos atopamos coa segunda cabeceira e aparece a excepción **StreamCorruptedException** e non poderemos ler máis obxectos.

O encabezado créase cada vez que se pon **new ObjectOutputStream(fichero)**. Para que non se engadan estas cabeceiras o que temos que facer redefinir a clase **ObjectOutputStream** creando unha nova que a herde (extends). E dentro desta clase redefinir o método **writeStreamHeader()** que é o que escribe os encabezados, e facemos que ese método non faga nada.

Entón, se o ficheiro xa foi creado, chamarase a ese método da clase redefinida.

A clase redefinida verase así:

```

import java.io.IOException;
import java.io.ObjectOutputStream;
import java.io.OutputStream;

/**
 * Redefinición da clase ObjectOutputStream para que non escriba unha
 * cabeceira
 * ao comezo do Stream.
 * Para non ter problemas coas cabeceiras dos obxectos e evitar o erro
 * StreamCorruptedException, creamos unha clase co noso propio
 * ObjectOutputStream, herdando do orixinal e redefinindo o método
 * writeStreamHeader() baleiro, para que non faga nada.
 */
public class MiObjectOutputStream extends ObjectOutputStream {
    /** Constructor que recibe OutputStream */
    public MiObjectOutputStream(OutputStream out) throws IOException {
        super(out);
    }

    /** Constructor sin parámetros */

```



```

    protected MiObjectOutputStream() throws IOException, SecurityException
    {
        super();
    }

    /** Redefinición do método de escribir a cabeceira para que non faga
nada. */
    protected void writeStreamHeader() throws IOException {
    }
}

```

E dentro do noso programa ao abrir o ficheiro para engadir novos obxectos preguntar se xa existe, se existe, o obxecto é creado coa clase redefinida e, se non existe, o ficheiro créase coa clase **ObjectOutputStream**:

```

import java.io.*;

public class EscribirFichObjectProblema {
    public static void main(String[] args) throws IOException {

        Persona persona; // defino variable persona

        File fichero = new File("FichPersona.dat"); // declara o ficheiro
        FileOutputStream fileout;
        /*
         * FileOutputStream fileout= new FileOutputStream(fichero,true);
        //crea o fluxo
         * de salida. Creamolo directamente
         * ao crear o ObjectOutputStream para que o ficheiro aínda non
        exista.
        */
        ObjectOutputStream dataOS;
        // conecta o fluxo de bytes ao fluxo de datos

        if (!fichero.exists()) { // Si el fichero no existe crea un
ObjectOutputStream, la primera vez
            dataOS = new ObjectOutputStream(new FileOutputStream(fichero,
true));
        } else { // Si xa existe o fichero crear un ObjectOutputStream
            // co método writeStreamHeader redefinido (sin facer nada)
            dataOS = new MiObjectOutputStream(new FileOutputStream(fichero,
true));
        }

        String nombres[] = { "Ana", "Luis Miguel", "Alicia", "Pedro",
"Manuel", "Andrés",
            "Julio", "Antonio", "María Jesús" };
    }
}

```

```
int edades[] = { 14, 15, 13, 15, 16, 12, 16, 14, 13 };
System.out.println("GRABO OS DATOS DE PERSONA.");
for (int i = 0; i < edades.length; i++) { // recorro os arrays
    persona = new Persona(nombres[i], edades[i]); // creo a persona
    dataOS.writeObject(persona); // escribo a persoa no ficheiro
    System.out.println("GRABO OS DATOS DE PERSONA.");
}
dataOS.close(); // pechar stream de saída
}
}
```

Bibliografía

Licenzas Consellería de Cultura, Educación e Ordenación Universitaria.

Materiais de FP a Distancia Consellería de Cultura, Educación e Ordenación Universitaria.

Manuais IES San Clemente.